

Department of Electrical and Information Engineering

University of Ruhuna

Mini Project Proposal

Machine Learning

EC5203

Machine Learning-Based Classification of Agricultural Market Price Decline Risk

Group 41 – AgroMaster

Ethipola K.J.R.R.W.M.R.E.M.W.P.T.B. (EG/2023/5589)

Geeganage S.P. (EG/2023/5607)

Nawodya T.H.Y. (EG/2023/5998)

Chamuditha K.D.T. (EG/2023/5537)

06 October 2026

Contents

Contents 1

1 Problem Statement 2

2 Objectives 3

3 Literature Review 4

4 Methodology 7

4.1 Overall approach . . . 7
4.2 Data acquisition and handling . . . 7
4.3 Data preprocessing . . . 7
4.4 Feature engineering . . . 8
4.5 Target definition and price-decline risk label . . . 9
4.6 Data splitting and validation . . . 9
4.7 Model development . . . 9
4.8 Evaluation metrics . . . 10
4.9 Model interpretation . . . 10
4.10 Tools and frameworks . . . 10

5 Dataset / Data Collection 11

6 Expected Outcomes 12

7 Percentage of Machine Learning Involvement 13

References 14

1. Problem Statement

Agricultural prices in India are highly volatile. Farmers choose what to plant and when to sell long before they know the price they will receive, and when many mandis receive the same crop at the same time, supply exceeds demand and prices collapse. Such oversupply episodes are common for perishable crops such as tomato and onion, leaving farmers selling below their cost of production while planning across the agri-food chain remains difficult [1].

Classical forecasting such as ARIMA models one price series at a time and handles non-linear seasonality, regional differences, and sudden shocks poorly [2]. Machine learning can learn these patterns jointly from millions of records [3], yet an explicit, decision-oriented price-decline risk indicator is seldom built on a nationwide mandi dataset.

This project addresses the problem by classifying agricultural market price-decline risk as Low, Medium or High seven calendar days ahead. Historical AGMARKNET prices, location information and engineered temporal features will be used to train and compare Random Forest and XGBoost classifiers. The predicted classes indicate future prices relative to historical price levels; they do not directly measure oversupply.

Back to Table of Contents

2. Objectives

The main goals of this mini project are:

- Build a reproducible preprocessing pipeline for AGMARKNET mandi price data and prepare a documented modelling subset.
- Engineer 18 model features (price statistics, lag and rolling-window features, and calendar/seasonal features) from the raw mandi data.
- Define a price-decline risk indicator (Low, Medium, High) for a seven-day horizon, using commodity-specific quantiles of the ratio between the future modal price and its trailing 90-day average.
- Develop supervised multi-class classification models (Random Forest and XGBoost) to predict the price-decline risk class, using class weights to handle the rarity of the High risk class.
- Compare both models against simple baselines (majority class, risk persistence, and logistic regression) using accuracy, precision, recall, macro F1, and the confusion matrix, with a leakage-free chronological split.
- Interpret the trained models with feature importance and SHAP values to identify which features drive price-decline risk.

Back to Table of Contents

3. Literature Review

Predicting agricultural commodity prices and market conditions has been studied for decades using statistical, machine learning, and deep learning methods. This section reviews six representative works that shape the classification approach proposed in this project. For each, we summarise the approach, its strengths and limitations, and how AgroMaster relates to it, and we then state the gap that the project addresses.

Study 1: Random Forests (Breiman). Random Forests build many decision trees, each trained on a bootstrap sample of the data and a random subset of features at every split, and then take a majority vote across the trees for classification [4]. The method captures non-linear relationships and feature interactions, is robust to noise and outliers, needs relatively little tuning, handles mixed numerical and categorical inputs, and provides out-of-bag error estimates and feature importance scores. Its limitations are that raw price levels drift upward with inflation over 25 years, which makes absolute price features less comparable across periods, and that large forests are memory-intensive on tens of millions of rows. Random Forests are also not temporal by nature, so they depend entirely on engineered lag and rolling features. We therefore use a Random Forest as the first supervised classifier, and we reduce the effect of price drift by defining the risk label from a ratio to the trailing average and by using relative features such as spread ratio and price momentum.

Study 2: Gradient boosting and XGBoost (Friedman; Chen and Guestrin). Gradient boosting builds trees one after another, each fitted to the errors of the current ensemble by following the gradient of a loss function [5]. XGBoost extends this idea with regularisation, sparsity-aware split finding, and a parallel, cache-efficient implementation that scales to large datasets [6]. Boosted trees are widely regarded as a leading choice for structured tabular data and are used in many winning solutions of data science competitions. Their weaknesses are a larger number of hyperparameters, a tendency to overfit noisy targets unless early stopping and regularisation are applied, and lower transparency than a single tree. We select XGBoost as the second classifier because of its accuracy and speed on large tabular data, and we tune it using time-aware validation and compare it directly with the Random Forest. Deep sequence models such as Long Short-Term Memory (LSTM) networks can learn temporal structure directly from raw sequences [7], but they need large amounts of data and compute, are sensitive to hyperparameters, and are hard to interpret when explaining a risk prediction to farmers or planners. LSTMs are therefore outside the core scope and are noted as possible future work.

Study 3: Statistical versus machine learning forecasting (Makridakis, Spiliotis and Assimakopoulos). This study compared a range of statistical and machine learning methods on a large collection of time series and reported that the machine learning methods tested were less accurate than the statistical ones, while stressing the value of

proper preprocessing and validation [3]. It also argued that machine learning forecasts should be judged against simple benchmarks and that uncertainty should be reported. Its strength is a careful, open comparison that warns against assuming complex models always win. Its limitation, from our perspective, is that it focuses on univariate series without rich external or categorical features. This motivates two design choices: every classifier in AgroMaster is compared with simple baselines, and the machine learning models are given engineered temporal and seasonal features rather than raw values only. The classical benchmark in this literature is ARIMA and its seasonal extension [2]. It is interpretable and works with modest data, but it is linear, assumes stationarity, is fitted to one series at a time, cannot easily use categorical information such as state, commodity or season, and gives a continuous forecast rather than a risk class. A separate ARIMA model for thousands of market–commodity series would also be impractical, so ARIMA is not used as a model here, and tree ensembles that learn jointly across all series are used instead.

Study 4: Evaluation of time series models (Bergmeir and Benítez; Cerqueira, Torgo and Mozetic). Standard random k-fold cross-validation assumes independent observations, which is false for time series. Bergmeir and Benítez showed that blocked and other adapted forms of cross-validation can be used safely for purely autoregressive predictors [8]. Cerqueira et al. compared performance estimation methods on many series and found that out-of-sample holdout approaches that respect time order are more reliable for non-stationary series, whereas cross-validation variants suit stationary series better [9]. Their main strength is practical guidance on how to avoid optimistic error estimates. Mandi prices are non-stationary because of inflation, policy changes, and weather shocks, so we will not use random splits. We use a chronological split into a development period and a final test period, together with five expanding-window chronological validation folds for tuning, and we compute all lag and rolling features strictly from past data.

Study 5: Machine learning for Indian vegetable price forecasting (Paul et al.). Paul et al. compared a generalized regression neural network, support vector regression, Random Forest, and gradient boosting against ARIMA for wholesale brinjal prices in 17 markets of Odisha, India [10]. The neural network performed best in all markets, and Random Forest was competitive in four. The strength of this work is a direct comparison with a statistical benchmark on Indian market data. Its limitations are a single commodity in one state and a focus on numeric price forecasts judged by error measures, rather than a decision-oriented risk class. AgroMaster extends this setting to many commodities and states and to a classification target.

Study 6: Price-state classification for agricultural commodities (Oktoviany, Knobloch and Korn). Oktoviany et al. proposed a two-step approach in which K-means clustering assigns price states, and K-nearest neighbours or Random Forest classification then predicts future states from external factors such as weather and economic

indicators, using corn futures data [11]. Its strength is treating price behaviour as discrete states that support risk management. Its limitations are the dependence on external-factor forecasts, the use of futures rather than physical mandi prices, and states that come from clustering rather than from a risk class defined for farmers. AgroMaster instead defines its classes from a ratio to the trailing average price and uses only price history and calendar features.

Gap addressed by this project. Much of the work reviewed above models one series at a time, or is evaluated on benchmark collections rather than on a very large agricultural market dataset. Work in this area also usually aims at predicting price values, without a decision-oriented class indicator such as price-decline risk. The AgroMaster project adds the following beyond existing work:

- A pipeline for a large agricultural market dataset, with modelling on a documented subset.
- An explicit price-decline risk target (Low, Medium, High) predicted directly by multi-class classifiers.
- Explicit handling of the strong class imbalance in price-decline risk through class weights and macro-averaged evaluation, so that the rare High risk class is not ignored.
- A leakage-free chronological evaluation that compares tree ensembles against simple baselines using standard classification metrics.
- Model interpretation with feature importance and SHAP values to show which signals precede price declines.

Back to Table of Contents

4. Methodology

4.1. Overall approach

The methodology follows a standard machine learning workflow with five stages: data acquisition, preprocessing, feature engineering, model development, and evaluation with interpretation. All stages are implemented in Python so that the full pipeline can be rerun from raw data to final results. The project has one supervised task: multi-class classification of price-decline risk as Low, Medium, or High for a commodity in a market seven calendar days ahead.

4.2. Data acquisition and handling

The dataset is the public AGMARKNET mandi price collection hosted on Kaggle [12]. Its native columns describe the location (state, district, market), the product (commodity, variety, grade), the arrival date, and three prices: minimum, maximum, and modal price. Because the data contains more than 75 million rows, it cannot be handled as a single in-memory table on a standard laptop. We will read it in chunks, convert it to a compressed columnar format such as Parquet, and use memory-efficient types (categorical codes and float32 values). Processing will use pandas with chunking or Polars. Initial modelling will use a subset of commodities and markets selected according to historical coverage and reporting completeness in the development period. The selected commodities, markets, date range and usable record count will be documented before model training. The source dataset's total size will be reported separately from the number of records used for modelling.

4.3. Data preprocessing

The following preprocessing steps will be applied before any modelling:

- Remove duplicate records and records with missing or non-positive prices.
- Parse the arrival date, group records by state, district, market, commodity, variety and grade, and align each series to calendar dates in time order. Multiple observations on the same date are aggregated using the median of each price field.
- Check price consistency (minimum $\leq$ modal $\leq$ maximum) and correct or drop records that violate it.
- Standardise category names (for example commodity and market spellings) so that the same product is not counted twice.
- Forward-fill missing predictor prices for at most three consecutive days, because mandi prices are not reported on every day. Longer gaps stay missing, and future target prices are never filled. Exclude records without an observed seven-day target or sufficient historical information.

- Check suspected outliers per commodity for data-entry errors, since a single error can distort rolling statistics, but retain genuine price crashes. Fit encoders, scalers and other learned preprocessing parameters using training data only.

4.4. Feature engineering

Feature engineering is a central part of the project. Starting from the native price and category columns, we construct 18 model features in three groups: price statistics, lag and rolling-window features, and calendar and seasonal features. Table 1 lists them. Lag and rolling features will be calculated separately for each state–district–market–commodity–variety–grade series. Lags will represent calendar days rather than row positions. All predictors will use information available at the prediction date. Tree models will use training-fitted ordinal encodings with explicit handling of unseen categories. The logistic regression baseline will use one-hot encoding for categorical features and standardisation of numerical features.

Table 1: Proposed 18 model features

	Feature	**Source**	**Type**	**Description**
1	Min_Price	Native	Numerical	Minimum price recorded for the day
2	Max_Price	Native	Numerical	Maximum price recorded for the day
3	Modal_Price	Native	Numerical	Most common (modal) price for the day
4	Commodity	Native	Categorical	Crop or commodity name
5	State	Native	Categorical	State in which the market is located
6	Price_Spread	Derived	Numerical	Max_Price − Min_Price
7	Spread_Ratio	Derived	Numerical	Price_Spread / Modal_Price
8	Modal_Lag_1	Derived	Numerical	Modal price 1 day earlier
9	Modal_Lag_7	Derived	Numerical	Modal price 7 days earlier
10	Modal_Lag_30	Derived	Numerical	Modal price 30 days earlier
11	MA_7	Derived	Numerical	7-day rolling mean of Modal_Price
12	MA_30	Derived	Numerical	30-day rolling mean of Modal_Price
13	Volatility_30	Derived	Numerical	30-day rolling standard deviation of Modal_Price
14	Price_Momentum	Derived	Numerical	7-day percentage change in Modal_Price
15	Month	Derived	Numerical	Month extracted from the arrival date
16	Year	Derived	Numerical	Year extracted from the arrival date
17	Day_of_Year	Derived	Numerical	Day number within the year (1–366)
18	Season	Derived	Categorical	Calendar season derived from a documented month-to-season mapping.

In total, 5 features come directly from the native columns and 13 are derived, giving 15 numerical and 3 categorical features. Lag and rolling features capture short-term momentum and volatility, while the calendar and season features capture recurring seasonal effects that may precede price declines. District, market, variety, and grade are used to identify and group series and are not used as direct inputs.

4.5. Target definition and price-decline risk label

The target is a three-class price-decline risk label for seven calendar days ahead. For each prediction date t, the ratio r_t is calculated by dividing the observed modal price at $t+7$ by the mean modal price over the 90 calendar days preceding t. Within each training fold, commodity-specific 10th and 30th percentile thresholds are estimated from these ratios. Ratios at or below the 10th percentile are labelled High risk; those above the 10th and at or below the 30th percentile are labelled Medium risk; the remainder are labelled Low risk. The thresholds are applied unchanged to the corresponding validation period. Future prices are used only to construct labels and never as predictors. Because the dataset records prices but not the quantity arriving at each market, the label is a price-based proxy and does not measure oversupply directly. High risk represents the lowest future-price ratios relative to the historical average; it does not necessarily indicate a decline from the current price. The proportion of High-risk cases with ratios below 1 will also be reported. Threshold choices will be justified with exploratory analysis and tested for sensitivity.

4.6. Data splitting and validation

The data will be divided chronologically into development and final test periods, for example development on 2001–2023 and final testing on 2024–2026, with the exact boundaries confirmed after exploratory analysis. Model selection will use five expanding-window chronological cross-validation folds within the development period: the development period is cut into six consecutive time blocks, and in fold k ($k = 1, \ldots, 5$) the first k blocks form the training set and block $k+1$ forms the validation set. All records sharing a date will be assigned to the same partition. Training observations whose seven-day target dates reach the validation period will be excluded. Preprocessing parameters, label thresholds and class weights will be estimated separately within each training fold. Hyperparameters will be selected using mean validation macro F1, with High-risk recall also reported. Random shuffling is not used, because it would let future information leak into training [8, 9]. The final test period will remain untouched until model selection is complete. After model selection, the selected model and preprocessing pipeline will be fitted on the development data. Commodity-specific label thresholds will be estimated from eligible development observations and applied unchanged to the final test period. Development observations whose target dates enter the test period will be excluded.

4.7. Model development

Three groups of models will be developed and compared:

- **Baselines.** A majority-class predictor, a risk-persistence baseline, and a regularised multinomial logistic regression provide reference points. The risk-persistence baseline will calculate the current price-to-historical-average ratio using information

available at the prediction date, apply the training-derived class thresholds, and predict that class for seven days ahead.

- **Random Forest.** Random Forest classification [4] is the first main supervised model. Key hyperparameters are the number of trees, maximum depth, and minimum leaf size, and balanced class weights are used.

- **Gradient boosting (XGBoost).** XGBoost classification [6] is the second main model. We tune learning rate, tree depth, number of estimators, and regularisation strength with randomised search and early stopping on the validation folds. Class weights handle the imbalance between Low, Medium, and High risk.

4.8. Evaluation metrics

Price-decline risk classifiers will be assessed with accuracy, per-class and macro-averaged precision, recall, and F1 score, and the confusion matrix. Because accuracy can be misleading when the Low class dominates, macro F1 and recall of the rare High risk class are the main comparison measures. Every model will be compared with the baselines, and results will be reported per commodity and per period, not only as a single overall figure.

4.9. Model interpretation

Feature importance from the tree models and SHAP values [13] will be used to explain which features drive predictions, for example whether recent momentum or seasonality matters more in particular commodities. Error analysis will examine serious misses, such as High risk periods predicted as Low risk, to understand where the models fail.

4.10. Tools and frameworks

The implementation uses Python with pandas, NumPy, and Polars for data handling, scikit-learn [14] for preprocessing, baselines, Random Forests, and metrics, XGBoost for gradient boosting, and Matplotlib, Seaborn, and Plotly for visualisation. Experiments run in Jupyter notebooks and Kaggle or Google Colab for extra compute, with Git for version control and shared work among the four team members.

Back to Table of Contents

5. Dataset / Data Collection

The project uses the public Kaggle dataset "Daily Market Prices of Commodity India (2001–2026)", sourced from AGMARKNET [12]. It contains more than 75,000,000 daily records in CSV form, with native columns for state, district, market, commodity, variety, grade, arrival date, and minimum, maximum, and modal prices, to be confirmed against the CSV header. Modelling will use a documented subset, reported separately. Feature engineering produces 18 features (15 numerical, 3 categorical). The data is public and contains no personal information. Quality is ensured through validation checks, outlier handling, and chronological validation.

Back to Table of Contents

6. Expected Outcomes

The project is expected to deliver:

- Python pipeline for cleaning and feature engineering.
- Trained price-decline risk classification models (Random Forest and XGBoost).
- Comparison of the classifiers against simple baselines, with per-commodity and per-period results.
- Metrics: accuracy, precision, recall, macro F1, confusion matrix.
- Model interpretation with feature importance and SHAP, plus code files and presentation.
- Comparative evaluation against majority-class, risk-persistence and logistic-regression baselines, focusing on macro F1 and High-risk recall.

Back to Table of Contents

7. Percentage of Machine Learning Involvement

More than 75% of the project is machine learning work. This covers data preprocessing, feature engineering, Random Forest and XGBoost training and tuning for classification, metric-based evaluation, and SHAP-based model interpretation. The remaining work (under 25%) is documentation and the presentation of results. Tools include scikit-learn, XGBoost, SHAP, Polars, pandas, and Python notebooks.

Back to Table of Contents

References

[1] O. Ahumada and J. R. Villalobos, "Application of planning models in the agri-food supply chain: A review," *European Journal of Operational Research*, vol. 196, no. 1, pp. 1–20, 2009.

[2] G. E. P. Box, G. M. Jenkins, and G. C. Reinsel, *Time Series Analysis: Forecasting and Control*. Hoboken, NJ, USA: Wiley, 4th ed., 2008.

[3] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," *PLoS ONE*, vol. 13, no. 3, p. e0194889, 2018.

[4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[5] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, (San Francisco, CA, USA), pp. 785–794, 2016.

[7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.

[9] V. Cerqueira, L. Torgo, and I. Mozetic, "Evaluating time series forecasting models: An empirical study on performance estimation methods," *Machine Learning*, vol. 109, pp. 1997–2028, 2020.

[10] R. K. Paul, M. Yeasin, P. Kumar, P. Kumar, M. Balasubramanian, H. S. Roy, A. K. Paul, and A. Gupta, "Machine learning techniques for forecasting agricultural prices: A case of brinjal in Odisha, India," *PLOS ONE*, vol. 17, no. 7, p. e0270553, 2022.

[11] P. Oktoviany, R. Knobloch, and R. Korn, "A machine learning-based price state prediction model for agricultural commodities using external factors," *Decisions in Economics and Finance*, vol. 44, no. 2, pp. 1063–1085, 2021.

[12] M. Khandelwal, "Daily market prices of commodity India (2001–2026)." Kaggle dataset, sourced from AGMARKNET, Government of India. [Online]. Available: https://www.kaggle.com/datasets/khandelwalmanas/daily-commodity-prices-india.

[13] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[14] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.